Air University

Operating Systems (Lab)

Year 2026

Ms. Fouzia Riaz

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: March 01, 2026

Quiz # 01

Question 1: Shell & Bash Scripting

Task Description

Create a shell script named: lab_task.sh

```
usama@ubuntu:~/Desktop$ touch lab_task.sh
usama@ubuntu:~/Desktop$ ls
lab_task.sh  new.txt  program  program1.cpp  program2.sh  secure_project
```

Your script should perform the following operations step-by-step.

Part 1: Script Setup & Comments

- Add a shebang line
- Add comments describing: Script purpose, your name, and Lab number

```
#!/bin/bash
# Script Purpose: OS Lab Quiz 1
# Name: Usam Arshad
# Lab Number: 04
```

Part 2: User Input & Variables

Ask the user to enter: Their name & age

- Store inputs using user-defined variables

```
usama@ubuntu:~/Desktop$ vi lab_task.sh
# --- Part 2: User Input & Variables ---
read -p "Enter your name: " user_name
read -p "Enter your age: " user_age
echo "Hello $user_name, you are $user_age years old."
```

- Display the entered information in a proper sentence

```
usama@ubuntu:~/Desktop$ chmod +x lab_task.sh
usama@ubuntu:~/Desktop$ ./lab_task.sh
Enter your name: Usama
Enter your age: 22
Hello Usama, you are 22 years old.
```

Part 3: Condition Check

- Write a Shell program to check if the given year is a leap year or not

```
# --- Part 3: Condition Check (Leap Y) ---
read -p "Enter a year: " year

if [ $((year % 400)) -eq 0 ]; then
    echo "$year is a leap year."
elif [ $((year % 100)) -eq 0 ]; then
    echo "$year is NOT a leap year."
elif [ $((year % 4)) -eq 0 ]; then
    echo "$year is a leap year."
else
    echo "$year is NOT a leap year."
fi
```

- **Result:**

```
usama@ubuntu:~/Desktop$ ./lab_task.sh
Enter a year: 2024
2024 is a leap year.
usama@ubuntu:~/Desktop$ ./lab_task.sh
Enter a year: 2025
2025 is NOT a leap year.
```

Part 4: String Comparison

- Ask the user to enter a single character (a or b)
- Use an if–else statement to:
  → Compare the entered character with a predefined value
  → Use string comparison, not numeric

```
# --- Part 4: String Comparison ---
read -p "Enter a character (a or b): " char
if [ "$char" == "a" ]; then
        echo "Match found!"
else
        echo "No match."
fi
```

- **Result:**

```
usama@ubuntu:~/Desktop$ ./lab_task.sh
Enter a character (a or b): c
No match.
usama@ubuntu:~/Desktop$ ./lab_task.sh
Enter a character (a or b): a
Match found!
```

Part 5: File Test Operator

- Ask the user to enter a file name
- Check whether the file:
  → Exists in the current directory
  → Display a message accordingly

```
# --- Part 5: File Test Operator ---
read -p "Enter a file name: " fname

if [ -e "$fname" ]; then
    echo "Success: The file '$fname' exists in the current directory."
else
    echo "Error: The file '$fname' was not found."
fi
```

- **Result:**

```
usama@ubuntu:~/Desktop$ ./lab_task.sh
Enter a file name: secureFile
Error: The file 'secureFile' was not found.
usama@ubuntu:~/Desktop$ ./lab_task.sh
Enter a file name: secure_project
Success: The file 'secure_project' exists in the current directory.
```

Part 6: Logical Operators

- Ask the user to enter their age
- Use AND (&&) or OR (||) to validate: Whether the age lies in a valid range
- Print an appropriate message

```
# --- Part 6: Logical Operators ---
read -p "Enter age for range check: " r_age
if [ "$r_age" -gt 0 ] && [ "$r_age" -lt 100 ]; then
    echo "Valid age."
else
        echo "Invalid Age."
fi
```

- **Result:**

```
usama@ubuntu:~/Desktop$ ./lab_task.sh
Enter age for range check: -1
Invalid Age.
usama@ubuntu:~/Desktop$ ./lab_task.sh
Enter age for range check: 24
Valid age.
```

Part 7: Arithmetic Operations

- Ask the user to enter two numbers
- Perform and display:
  → Addition
  → Subtraction
  → Multiplication
  → Division
- Use arithmetic expansion

```
# --- Part 7: Arithmetic Operations ---
read -p "Enter first number: " n1
read -p "Enter second number: " n2

sum=$((n1 + n2))
diff=$((n1 - n2))
prod=$((n1 * n2))

echo "Sum:            $sum"
echo "Difference:     $diff"
echo "Product:        $prod"

# Check for division by zero
if [ $n2 -ne 0 ]; then
    div=$((n1 / n2))
    rem=$((n1 % n2))
    echo "Division:       $div (Remainder: $rem)"
else
    echo "Error: Division by zero is not allowed."
fi
```

- **Result:**

```
usama@ubuntu:~/Desktop$ ./lab_task.sh
Enter first number: 12
Enter second number: 3
Sum:            15
Difference:     9
Product:        36
Division:       4 (Remainder: 0)
```

Part 8: Command Line Arguments

- Accept two arguments from the command line
- Display:
  → Script name
  → First argument
  → Second argument
  → Total number of arguments

```
# --- Part 8: Command Line Arguments ---
echo -e "Script: $0 \nArgs: $1, $2 \nCount: $#"
```

- **Results:**

```
usama@ubuntu:~/Desktop$ ./lab_task.sh Usama Quiz1
Script: ./lab_task.sh
Args: Usama, Quiz1
Count: 2
```

Part 9: Case Statement

- Take one command-line argument representing a subject name
- Use a case statement to display the subject category
- Display a default message for an invalid subject

Example idea: OS → Core Subject Math → Supporting Subject

```
# --- Part 9: Case Statement ---
subject=$1

case $subject in
    "OS" | "os" | "Operating System")
        echo "Subject: $subject -> Category: Core Subject"
        ;;
    "Math" | "math" | "Calculus")
        echo "Subject: $subject -> Category: Supporting Subject"
        ;;
    *)
        echo "Result: '$subject' is not a recognized subject."
        ;;
esac
```

- **Result:**

```
usama@ubuntu:~/Desktop$ ./lab_task.sh Math
Subject: Math -> Category: Supporting Subject
usama@ubuntu:~/Desktop$ ./lab_task.sh OS
Subject: OS -> Category: Core Subject
usama@ubuntu:~/Desktop$ ./lab_task.sh Networks
Result: 'Networks' is not a recognized subject.
```

Part 10: Arrays

- Ask the user to enter multiple values into an array
- Display:
  → First element
  → All elements
  → Total number of elements

```
# --- Part 10: Arrays ---
read -a my_array -p "Enter multiple values (space separated): "
echo -e "First: ${my_array[0]} \nAll: ${my_array[@]} \nTotal: ${#my_array[@]}"
```

- **Result:**

```
usama@ubuntu:~/Desktop$ ./lab_task.sh
Enter multiple values (space separated): 1 2 3 4 5 6 7 8 9
First: 1
All: 1 2 3 4 5 6 7 8 9
Total: 9
```

Part 11: Loops

a) **While Loop:** Print numbers from 1 to 10
b) **For Loop:** Print all files in the current directory
c) **Loop Control**

Use:

- break at a specific condition
- continue under another condition

```
# --- Part 11: Loops ---
echo "While Loop 1-10 (Skip 5, Break at 9):"
i=1
while [ $i -le 10 ]; do
    if [ $i -eq 9 ]; then break; fi
    if [ $i -eq 5 ]; then i=$((i+1)); continue; fi
    echo -n "$i "
    i=$((i+1))
done
echo -e "\nFor-Loop \nFiles:"
for f in *; do echo "File: $f"; done
```

- **Result:**

```
usama@ubuntu:~/Desktop$ ./lab_task.sh
While Loop 1-10 (Skip 5, Break at 9):
1 2 3 4 6 7 8
For-Loop
Files:
File: 1
File: lab_task.sh
File: new.txt
File: program
File: program1.cpp
File: program2.sh
File: secure_project
```

Part 12: Function

Create a function that:

- Accepts one argument
- Prints a meaningful message using that argument
- Call the function at least twice

```
# --- Part 12: Function ---
greet() { echo "Function says: Hello $1"; }
greet "Student"
greet "Instructor"
```

- **Result:**

```
usama@ubuntu:~/Desktop$ ./lab_task.sh
Function says: Hello Student
Function says: Hello Instructor
```

Complete Script:

```
#!/bin/bash
# Script Purpose: OS Lab Quiz 1
# Name: Usam Arshad
# Lab Number: 04

# --- Part 2: User Input & Variables ---
read -p "Enter your name: " user_name
read -p "Enter your age: " user_age
echo "Hello $user_name, you are $user_age years old."

# --- Part 3: Condition Check (Leap Y) ---
read -p "Enter a year: " year

if [ $((year % 400)) -eq 0 ]; then
    echo "$year is a leap year."
elif [ $((year % 100)) -eq 0 ]; then
    echo "$year is NOT a leap year."
elif [ $((year % 4)) -eq 0 ]; then
    echo "$year is a leap year."
else
    echo "$year is NOT a leap year."
fi

# --- Part 4: String Comparison ---
read -p "Enter a character (a or b): " char
if [ "$char" == "a" ]; then
        echo "Match found!"
else
        echo "No match."
fi

# --- Part 5: File Test Operator ---
read -p "Enter a file name: " fname

if [ -e "$fname" ]; then
    echo "Success: The file '$fname' exists in the current directory."
else
    echo "Error: The file '$fname' was not found."
fi


# --- Part 6: Logical Operators ---
read -p "Enter age for range check: " r_age
if [ "$r_age" -gt 0 ] && [ "$r_age" -lt 100 ]; then
    echo "Valid age."
else
        echo "Invalid Age."
fi
```

```
# --- Part 7: Arithmetic Operations ---
read -p "Enter first number: " n1
read -p "Enter second number: " n2

sum=$((n1 + n2))
diff=$((n1 - n2))
prod=$((n1 * n2))

echo "Sum:            $sum"
echo "Difference:     $diff"
echo "Product:        $prod"

# Check for division by zero
if [ $n2 -ne 0 ]; then
    div=$((n1 / n2))
    rem=$((n1 % n2))
    echo "Division:       $div (Remainder: $rem)"
else
    echo "Error: Division by zero is not allowed."
fi


# --- Part 8: Command Line Arguments ---
echo -e "Script: $0 \nArgs: $1, $2 \nCount: $#"

# --- Part 9: Case Statement ---
subject=$1

case $subject in
    "OS" | "os" | "Operating System")
        echo "Subject: $subject -> Category: Core Subject"
        ;;
    "Math" | "math" | "Calculus")
        echo "Subject: $subject -> Category: Supporting Subject"
        ;;
    *)
        echo "Result: '$subject' is not a recognized subject."
        ;;
esac

# --- Part 10: Arrays ---
read -a my_array -p "Enter multiple values (space separated): "
echo -e "First: ${my_array[0]} \nAll: ${my_array[@]} \nTotal: ${#my_array[@]}"

# --- Part 11: Loops ---
echo "While Loop 1-10 (Skip 5, Break at 9):"
i=1
while [ $i -le 10 ]; do
    if [ $i -eq 9 ]; then break; fi
    if [ $i -eq 5 ]; then i=$((i+1)); continue; fi
    echo -n "$i "
    i=$((i+1))
done
echo -e "\nFor-Loop \nFiles:"
for f in *; do echo "File: $f"; done

# --- Part 12: Function ---
greet() { echo "Function says: Hello $1"; }
greet "Student"
greet "Instructor"
```

THE END